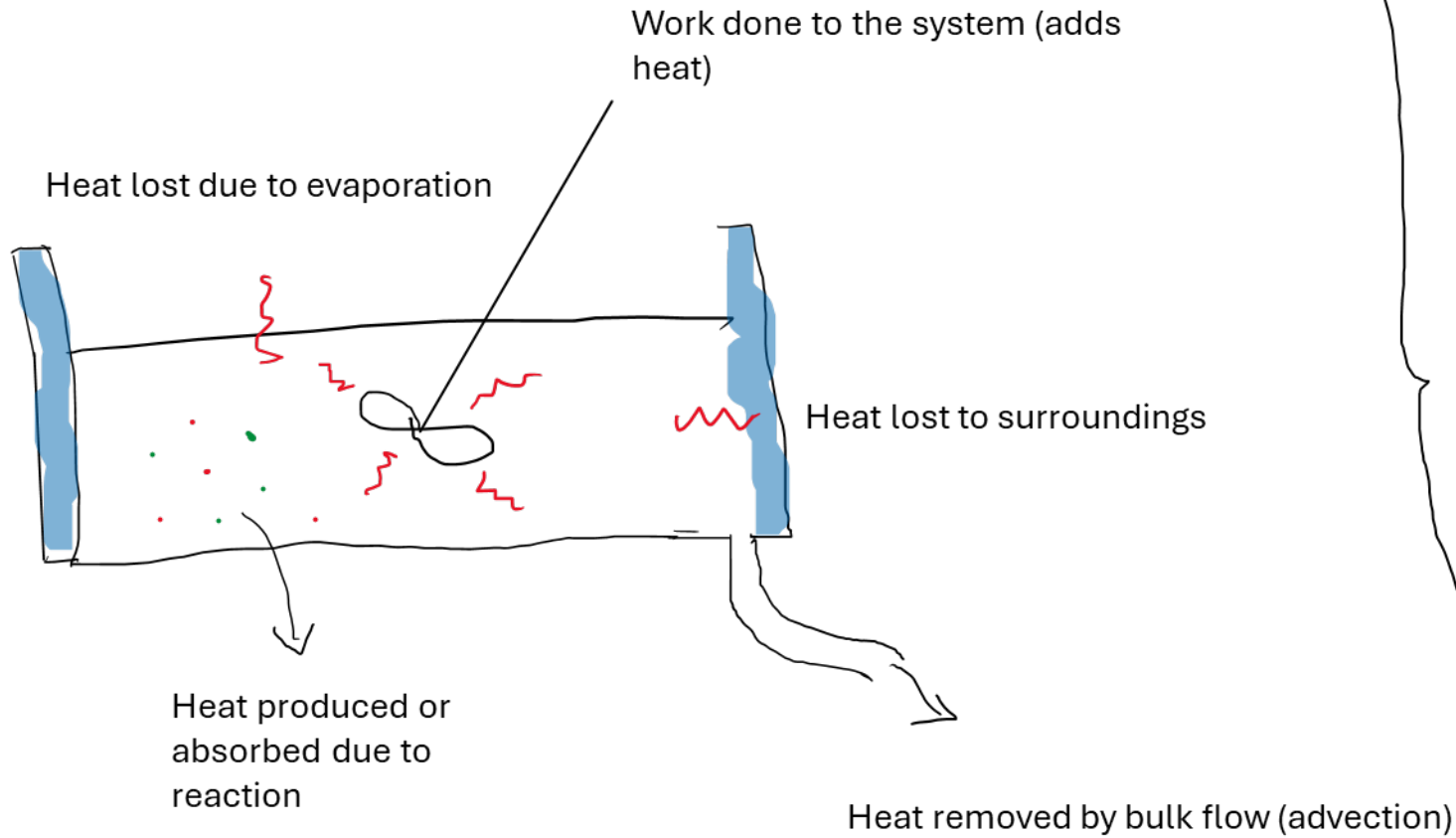


Coupling mass and energy balance

Energy dynamics

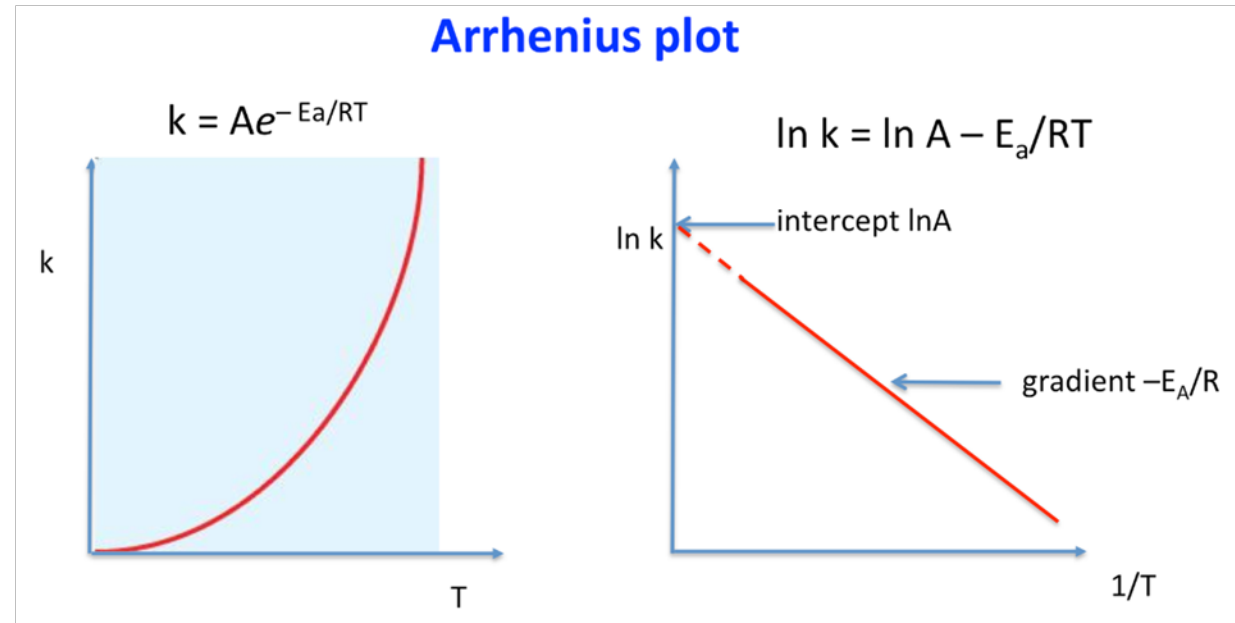
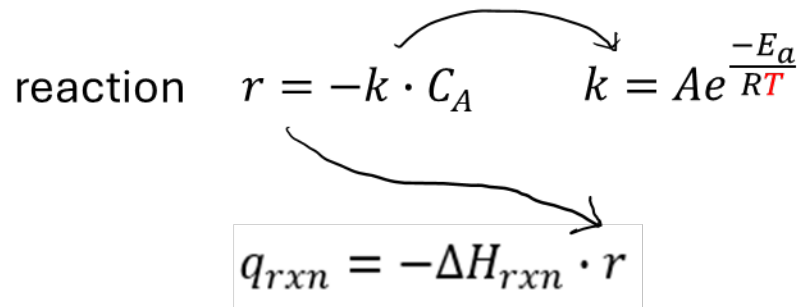


Temperature 

$$mC_p \frac{dT}{dt} = Q_{in} - Q_{out} + S$$

$$\frac{dT}{dt} = \frac{Q_{in} - Q_{out} + S}{mC_p}$$

Mass balance linkage to temperature



A = pre-exponential factor
 E_a = Activation energy (positive)

Other links:

diffusion coefficient, mass transfer coefficient, pressure (in gas phase), solubility, equilibrium constant..etc

$$D_{AB} = \frac{13.26 \cdot 10^{-5}}{\eta^{1.14} V_i^{0.589}}$$

$$k_T \cdot (C_L - C_{bulk}), k_T = f(T, \dots)$$

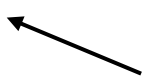
$$H(T) = H^\circ e^{\left[\frac{-\Delta_{sol}H}{R} \left(\frac{1}{T} - \frac{1}{T^\circ} \right) \right]}$$

$$\eta = f(T)$$

- $\Delta H_{rxn} < 0$ if energy released (exothermic)
- $\Delta H_{rxn} > 0$ if energy absorbed (endothermic)

$$q_{rxn} = -\Delta H_{rxn} \cdot r$$

positive



Discuss with classmate

How does an irreversible exothermic reaction affect q_{rxn} and r and the system as a whole?

How does an irreversible endothermic reaction affect q_{rxn} and r and the system as a whole?

Batch reactor, first order reaction, tracing only reactant A, where $A \rightarrow B$ and the reaction generates energy (exothermic reaction!)

mass balance of A

Conservation equation:

$$acc = \cancel{in} - \cancel{out} + \cancel{generated} - consumed$$

Constitutive equation:

$$r = k \cdot C_A$$

$$k = Ae^{\frac{-E_a}{RT}}$$

$$r: \frac{\text{mass}}{\text{volume time}}, k: \frac{1}{\text{time}}, C_A: \frac{\text{mass}}{\text{volume}}$$

Governing equation:

$$\frac{dM_A}{dt} = -rV$$

$$\frac{dM_A}{dt}: \frac{\text{mass}}{\text{time}}$$

energy balance

Conservation equation:

$$acc = \cancel{in} - \cancel{out} + generated - \cancel{consumed}$$

Constitutive equation:

$$q_{rxn} = -\Delta H_{rxn} \cdot r$$

$$q_{rxn}: \frac{J}{\text{volume time}}, \Delta H_{rxn}: \frac{J}{\text{mass}}, r: \frac{\text{mass}}{\text{volume time}}$$

Governing equation:

$$m \cdot C_p \frac{dT}{dt} = q_{rxn}V$$



$$\Delta H_{rxn} < 0 \text{ if energy released}$$

$$\Delta H_{rxn} > 0 \text{ if energy absorbed}$$

Tip: Sometimes generated and consumed is just condensed to "S", which stands for "source".

S basically represents all the stuff that is not from heat advection and heat diffusion!

Example: heat removed or added from jacket, agitation, evaporation, heat from reaction, heat of fusion...

In 0D:

Mass balance of A + heat balance $\rightarrow$ 2 state variables

Pseudo code

```
rates(t, y, V, other_args):  
  
    mA = y[0]  
    T = y[1]  
  
    cA = mA/V  
    m = V * rho  
    k = A * np.exp(-Ea/(R*T))  
    r = -k * cA  
  
    qrxn = -delta_H * r  
  
    dmAdt = r*V  
    dTdt = qrxn * V/(m * Cp)  
  
    return np.array([dmAdt, dTdt])
```

In 1D : Mass balance of A at x positions + heat balance at x positions → two state variables x positions (mini project 2).

Mass balance

Governing equation:

$$\frac{\partial C_A}{\partial t} = D \frac{\partial^2 C_A}{\partial x^2} - v \frac{\partial C_A}{\partial x} - r$$

Constitutive equation:

$$r = k \cdot C_A$$

$$k = A e^{\frac{-E_a}{RT}}$$

Governing equation final

$$\frac{\partial C_A}{\partial t} = D \frac{\partial^2 C_A}{\partial x^2} - k \cdot C_A$$

No heat
advection



Energy balance

Conservation equation:

$$\rho C_p \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2} - v \frac{\partial T}{\partial x} + S$$

Constitutive equation:

$$S = q_{rxn} = -\Delta H_{rxn} \cdot r$$

Governing equation:

$$\frac{\partial T}{\partial t} = \frac{k}{\rho C_p} \frac{\partial^2 T}{\partial x^2} + \frac{q_{rxn}}{\rho C_p}$$



Discuss with a classmate

You have 3 chemical species and a heat balance that you want to track in a model. You remember this from a quick glitch at the code.

```
x_grid = np.arange(0, 1 + 0.1, 0.1) # meters
```

```
tmax = 100 # seconds
```

```
sol = solve_ivp(.....t_eval = np.linspace(0, tmax, 50))
```

What is the dimensions of the sol.y? (approximately)

What does sol.y[16, 25] most likely represent? (approximately)

Pseudo code (without BCs)

```
rates_1D(t, y, other_args...):
```

```
    Ca = y[:len(x_grid)]
```

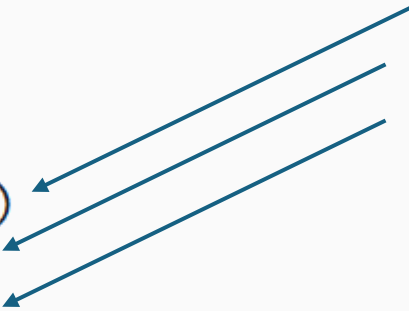
```
    T = y[len(x_grid):]
```

```
    k = A*np.exp(-Ea/(R*T))
```

```
    r = -k * Ca
```

```
    qrxn = -delta_H * r
```

Could also be in loop with indexing directly



```
    for i in range(1, (len(x_grid)-1)):
```

```
        dCadt[i] = D * (Ca[i-1] - 2*Ca[i] + Ca[i+1])/dx**2 + r[i]
```

```
        dTdt[i] = k/(Cp * ro) * (T[i-1] - 2*T[i] + T[i+1])/dx**2 + qrxn[i]/(Cp * ro)
```

```
    return np.concatenate([dCadt, dTdt])
```

The 1D version is on concentration basis.

Integration (e.g. trapz method over space) of output gives you the total mass in system!

(Like in mini project2)